UNIVERSAL CODE SCANNER

06 · Explicación profunda del código

Documentación de sistema

Versión analizada: **1.1.0+2**
Commit: **6d5e0b7**
Fecha del documento: **2026-08-27**

Documento generado desde el Markdown de **docs/system-documentation/** con **tool/build_system_documentation_pdf.py**. El Markdown es la fuente: no editar este PDF.

Contenido

1. 1 · Arranque: `main` → `AppServices`
2. 2 · Lectura con cámara
3. 3 · Lotes de imágenes y de PDF
4. 4 · Interpretación del contenido
5. 5 · Cifrado y persistencia
6. 6 · Rotación de llave
7. 7 · Inventario continuo
8. 8 · Recuperación
9. 9 · Generador
10. 10 · Exportación
11. 11 · Pantallas resumidas
12. 12 · Lo que un lector nuevo descubriría a golpes

Este documento explica **flujo por flujo** qué ocurre dentro del sistema, por qué está escrito así y qué casos límite contempla. La referencia seca de firmas está en 05-technical-reference.md; aquí se explica el razonamiento.

Se explican en detalle los flujos que concentran las decisiones. Las pantallas de lista y formulario —historial, generador, formatos— se resumen: son voluminosas y poco densas en decisiones.

---

1 · Arranque: `main` → `AppServices`

1.1 `main.dart`

Catorce líneas que hacen tres cosas antes de nada más:

```
WidgetsFlutterBinding.ensureInitialized();
installGlobalErrorHandlers();
runZonedGuarded<void>(() => runApp(const BootstrapHost()), …);
```

El orden importa. `ensureInitialized()` debe ir primero porque `installGlobalErrorHandlers()` toca `PlatformDispatcher.instance`. Y `runZonedGuarded` envuelve a `runApp`, no al revés: una excepción asíncrona que escapa de un `Future` sin `catch` cae en la zona, no en `FlutterError.onError`.

Los tres manejadores capturan tres clases distintas de fallo, y por eso existen los tres:

Manejador	Captura	Área
`FlutterError.onError`	Errores del árbol de widgets, durante `build`, `layout` o `paint`	`flutter`
`PlatformDispatcher.instance.onError`	Errores asíncronos que llegan del motor	`platform`
El de `runZonedGuarded`	Todo lo demás que escape de la zona	`root_zone`

`FlutterError.onError` **vuelve a llamar a `FlutterError.presentError(details)`** después de registrar. Sin eso, se perdería el volcado rojo en depuración, que es donde un desarrollador espera verlo.

1.2 `AppBootstrapper.initialize`

El grafo se construye en un orden que no es arbitrario:

```
database = temporary ? await AppDatabase.openTemporary() : await AppDatabase.open();
final RecoveryRepository recovery = RecoveryRepository(database);
final EncryptionMetadataRepository encryptionMetadata = EncryptionMetadataRepository(database);
final String activeKeyId = temporary ? PayloadCipher.currentKeyId : await encryptionMetadata.loadActiveKey
final PayloadCipher cipher = PayloadCipher(keyProvider: temporary ? MemoryEncryptionKeyProvider() : null,
```

La llave activa se lee de la base antes de construir el cifrador. Si se hiciera al revés, el cifrador arrancaría con `'v2'` y la primera escritura tras una rotación previa crearía una llave nueva bajo un identificador equivocado. `loadActiveKeyId` es además el punto donde ocurre la migración desde preferencias: si el almacén `_security_meta` está vacío, lee la preferencia `encryption_active_key_id`, la escribe en la base y **la borra de preferencias**.

Después:

```
await Future.wait(<Future<void>>[scans.initialize(), inventory.initialize()]);
await scans.pruneOlderThan(settings.value.historyRetentionDays);
```

Historial e inventario se inicializan en paralelo porque no dependen entre sí. La poda va después porque necesita el valor de `historyRetentionDays`, que viene de las preferencias ya inicializadas.

El bloque completo está en un `try` cuyo `catch` hace:

```
} on Object {
 await database?.close();
 rethrow;
}
```

Es decir: **si algo falla a mitad, la base se cierra antes de propagar**. Sin eso, el reintento desde la pantalla de inicio seguro intentaría abrir un archivo ya bloqueado por la propia aplicación.

En modo temporal, `settings.initialize()` va dentro de su propio `try/catch` vacío. El comentario del código lo dice: «Temporary mode remains available even when preferences are unavailable». Es la última red de seguridad: si lo que falló fue el propio subsistema de preferencias, el modo temporal debe seguir arrancando con `const AppSettings()` por defecto.

1.3 La pantalla de inicio seguro

`_BootstrapHostState._retry` hace algo que parece redundante y no lo es:

```
_future = AppBootstrapper.initialize(temporary: temporary).catchError((error, stack) {
 AppDiagnostics.instance.record(error, stack, area: temporary ? 'bootstrap_temporary' : 'bootstrap');
 _failure = StartupFailure.from(error, stack);
 throw error;
});
```

Registra el fallo, guarda `_failure` **y vuelve a lanzar**. El relanzamiento mantiene el `Future` en estado de error, de modo que el `FutureBuilder` no se quede colgado en el indicador de progreso. `build` comprueba `_failure` primero —porque `_failure` se asigna fuera de un `setState`— y el `FutureBuilder` comprueba `snapshot.hasError && _failure != null` como segunda vía.

Nótese que `_failure = …` ocurre sin `setState`. Funciona porque el propio `setState` que envuelve la asignación de `_future` ya programó una reconstrucción, y `catchError` se resuelve en un microtask posterior que dispara la reconstrucción del `FutureBuilder`. **Es frágil**: si el orden cambiara, la pantalla podría quedarse en el indicador de progreso. R-17.

---

2 · Lectura con cámara

2.1 Por qué la pantalla gestiona el ciclo de vida

`MobileScanner` solo atiende el ciclo de vida cuando **él mismo** crea el controlador. Estas pantallas le pasan el suyo —lo necesitan para el zoom, la linterna y `analyzeImage`—, así que la gestión les corresponde. El comentario del código lo dice sin rodeos: sin esto, volver del segundo plano o del diálogo de permiso del sistema dejaba la vista previa congelada sin más salida que abandonar la pestaña.

```
case AppLifecycleState.resumed:
 if (!_paused && !_handlingResult) unawaited(_startCamera());
case AppLifecycleState.inactive:
 unawaited(_stopCamera());
```

Las dos condiciones de `resumed` importan. `!_paused` respeta la decisión explícita del usuario de pausar; `!_handlingResult` evita que volver del diálogo de compartir reanime la cámara mientras la hoja de resultados sigue abierta.

`hidden`, `paused` y `detached` no hacen nada porque `inactive` **siempre precede** a los tres en Android e iOS: parar dos veces sería redundante.

2.2 Toda llamada al controlador va envuelta

```
Future<void> _startCamera() async {
 try {
   await _engine.start();
   if (mounted) setState(() => _startFailure = null);
 } on MobileScannerException catch (error) {
   if (mounted) setState(() => _startFailure = _describe(error));
 } on Object {
   if (mounted) setState(() => _startFailure = 'No fue posible iniciar la cámara.');
 }
}
```

El comentario del código explica el porqué: «Every call into the controller can throw: it may be disposed, still starting, or not yet attached to a widget. A failure here must leave the screen in a state the user can act on, never in an exception.»

`_describe` traduce dos códigos concretos a instrucciones accionables (`permissionDenied` → «Actívalo en los ajustes del sistema»; `unsupported` → «Este dispositivo no permite leer códigos con la cámara») y el resto a «Toca "Reintentar"». El texto genérico existe porque `MobileScannerErrorCode` puede crecer con el paquete.

`_stopCamera` captura y **no hace nada**: parar una cámara ya parada o ya desechada no es un error en este contexto.

2.3 «Reiniciar cámara»: el orden que importa

```
await _stopCamera();
try { await _engine.dispose(); } on Object { }
if (!mounted) return;
setState(() {
 _createEngine();
 _previewKey = UniqueKey();
 …
});
```

El comentario original: «The old controller is released before the new one exists: both share a single platform camera session, and disposing it afterwards would tear down the session the new controller has just claimed.»

`_previewKey = UniqueKey()` es la otra mitad. Sin cambiar la clave, Flutter reutilizaría el elemento `MobileScanner` existente y lo dejaría atado al controlador viejo. Un controlador que no arrancó **no se puede reutilizar**: la recuperación consiste literalmente en construir uno nuevo.

2.4 Cómo se decide la fase visible

```
ScanPhase _phaseFor(MobileScannerState state) {
 if (_startFailure != null || state.error != null) return ScanPhase.unavailable;
 if (_paused || _handlingResult) return ScanPhase.paused;
 if (state.isRunning) return ScanPhase.scanning;
 return ScanPhase.starting;
}
```

El orden es una precedencia: el fallo gana sobre la pausa, y la pausa sobre el estado real del motor. Que `_handlingResult` produzca `paused` es lo que hace que la barra deje de moverse mientras se muestra la hoja de resultados, aunque técnicamente la cámara ya se detuvo.

`ScanStatusBar` traduce la fase a título, icono, color y —lo importante— al valor del `LinearProgressIndicator`:

```
value: busy && animate ? null : (busy ? 1 : 0)
```

`null` es la barra indeterminada que se mueve. Un valor fijo la dibuja quieta. Así, con movimiento reducido activado, una cámara ocupada se dibuja llena (valor `1`) y una en pausa vacía (valor `0`): el estado sigue siendo legible **sin animación**. Hay cinco pruebas de widget sobre exactamente este comportamiento.

2.5 El marco de lectura, y por qué no tiene umbral

```
MobileScanner(
 key: _previewKey,
 controller: _engine.controller,
 scanWindow: _settings.useScanWindow ? window : null,
 // No threshold: …
 tapToFocus: true,
 onDetect: (capture) => unawaited(_handleCapture(capture, source: 'Cámara')),
 errorBuilder: (context, error) => _ScannerError(error: error, onRetry: _restartCamera),
)
```

El comentario que ocupa el lugar del parámetro ausente es el registro del defecto que originó la versión 1.1.0: la ventana de lectura se expresa en porcentajes de 0 a 1 y el umbral estaba en `4`, es decir, «no recalcular nunca». Si el primer cálculo salía mal —porque el tamaño de la cámara y la orientación pueden llegar después del primer cuadro—, **todos los códigos quedaban fuera de la región analizada**. La corrección fue quitar el umbral.

El rectángulo es cuadrado y ocupa el 68 % del lado menor, centrado (`LayoutBuilder` → `Rect.fromCenter`). Cuando `useScanWindow` está desactivado se pasa `null` y se analiza toda la imagen: es la salida que ofrece el README cuando un código no entra.

2.6 `_handleCapture`: deduplicación en dos niveles

```
if (_handlingResult || _paused || !mounted) return;
final Map<String, Barcode> unique = <String, Barcode>{};
for (final Barcode barcode in capture.barcodes) {
 final String raw = ScanRecord.payloadForBarcode(barcode);
 if (raw.isNotEmpty) unique.putIfAbsent(raw, () => barcode);
}
if (unique.isEmpty) return;
```

Primer nivel: dentro de una misma captura, dos detecciones del mismo texto producen un solo registro. `putIfAbsent` conserva la primera.

```
final List<String> signature = unique.keys.toList()..sort();
final String joined = signature.join('|');
if (_lastSignature == joined && _lastDetectedAt != null &&
   now.difference(_lastDetectedAt!) < const Duration(seconds: 2)) return;
```

Segundo nivel: entre capturas, la misma **combinación** de códigos dentro de 2 segundos se ignora. La firma se ordena antes de unir para que el orden en que el motor devuelve los códigos no genere firmas distintas del mismo conjunto.

Esto es necesario aunque el controlador use `DetectionSpeed.noDuplicates`, porque ese modo se reinicia con cada `start()` y la pantalla arranca y para la cámara constantemente.

Después:

```
try {
 await _stopCamera();
 await _feedback.success(sound: …, vibration: …);
 final records = unique.values.map((b) => ScanRecord.fromBarcode(b, source: source, scannedAt: now)).toLis
 await _persistAndShow(records);
} finally {
 if (mounted) setState(() => _handlingResult = false);
 if (mounted && !_paused) await _startCamera();
}
```

La cámara se detiene **antes** de la confirmación sonora y del análisis: sin eso, seguiría entregando cuadros mientras se construye la hoja de resultados, y cada uno volvería a entrar en `_handleCapture`. El `finally` garantiza que la cámara vuelve aunque `_persistAndShow` lance.

Todos los registros de una misma captura comparten `scannedAt: now`. Sus `id` siguen siendo distintos porque el `rawValue` difiere.

2.7 `_persistAndShow`: dónde se decide qué se guarda

```
if (_settings.saveHistory && !_settings.privateMode) {
 final persistent = records.where((r) => !r.isSensitive).toList(growable: false);
 await widget.store.addAll(persistent);
}
…
await showModalBottomSheet<void>(… ScanResultsSheet(records: records, …));
```

Tres condiciones se combinan con **Y**: el ajuste global `saveHistory`, la ausencia de `privateMode`, y que el registro concreto no sea sensible. Nótese que la hoja de resultados recibe **`records`, no `persistent`**: lo sensible se muestra, simplemente no se guarda.

`isSensitive` es `parsed.sensitive`, que el intérprete pone a `true` para OTP, criptomonedas, los tres formatos de pago, AAMVA y **Wi-Fi solo si trae contraseña**. Un QR de red abierta sí se guarda.

---

3 · Lotes de imágenes y de PDF

3.1 El diálogo de progreso, y por qué se cierra así

`_runBatchDialog` resuelve un problema concreto: mostrar un diálogo modal mientras corre una operación asíncrona, y cerrarlo con seguridad tanto si termina como si falla.

```
bool dialogOpen = true;
final NavigatorState navigator = Navigator.of(context, rootNavigator: true);
final Future<void> dialog = showDialog<void>(…).whenComplete(() => dialogOpen = false);
await Future<void>.delayed(Duration.zero);
try {
 return await operation();
} finally {
 if (dialogOpen && navigator.mounted) navigator.pop();
 await dialog;
 progress.dispose();
 token.dispose();
}
```

Cuatro decisiones:

1. **`Navigator.of` se captura antes** de la operación. Después, el `context` podría estar desmontado y `Navigator.of(context)` lanzaría.
2. **`Future.delayed(Duration.zero)`** cede el turno para que el diálogo llegue a construirse antes de que empiece el trabajo. Sin eso, una operación rápida podría intentar cerrar un diálogo que aún no existe.
3. **`dialogOpen`** evita un `pop` de más si el diálogo ya se cerró por otra vía.
4. **`await dialog`** espera a que la animación de cierre termine antes de liberar `progress` y `token`, que el propio diálogo sigue escuchando.

3.2 Galería

```
final List<XFile> images = await _imagePicker.pickMultiImage(limit: 20);
```

El límite de 20 es del selector, no de la aplicación: el sistema operativo no deja seleccionar más.

Dentro del lote, por cada imagen: `token.throwIfCancelled()`, actualizar progreso, `_engine.analyzeImage(path)`, deduplicar por texto dentro de la imagen, y `await Future<void>.delayed(Duration.zero)` para ceder el turno al bucle de eventos —sin eso, el diálogo no se repintaría y el botón «Cancelar» no respondería.

El `finally` interno reanuda la cámara si procede; el externo distingue tres finales: cancelación (mensaje explícito de que **no se modificó el historial**), cualquier otro error, y éxito con cero códigos.

3.3 PDF

El caso del PDF tiene una complicación adicional: el trabajo son **dos fases** —renderizar páginas a PNG y después analizar esos PNG—, y el progreso debe reflejarlo. La solución es contar sobre `total * 2`:

```
onProgress: (current, total) => progress.update(
 label: 'Renderizando página $current de $total',
 current: current,
 total: total * 2,
),
```

y en la segunda fase `current: pages.length + index`. Así la barra recorre la mitad izquierda renderizando y la derecha analizando.

`PdfPageRenderer.pickAndRender` merece leerse con cuidado:

```
final double scale = _renderScale(page.width, page.height);
final PdfPageRenderCancellationToken renderToken = page.createCancellationToken();
void cancelRender() {
 if (cancellationToken?.isCancelled ?? false) renderToken.cancel();
}
cancellationToken?.addListener(cancelRender);
cancelRender();
try {
 pdfImage = await page.render(…, cancellationToken: renderToken);
} finally {
 cancellationToken?.removeListener(cancelRender);
}
```

La cancelación se propaga a **dos niveles**: el bucle del lote y el renderizador de la página. `cancelRender()` se llama una vez inmediatamente después de suscribir, para cubrir el caso de que el usuario ya hubiera cancelado antes de que se registrara el *listener*. Y el `removeListener` en `finally` evita acumular suscripciones página a página.

`_renderScale` acota la escala a `[1.5, 4.0]` apuntando a 2400 px en el lado mayor. Es el compromiso entre legibilidad de un código pequeño dentro de una página y memoria: un A4 a 4× son unos 3400×4800 píxeles.

La limpieza es exhaustiva. Hay tres caminos:

- **éxito con cero páginas útiles**: se borra el directorio temporal;
- **excepción**: `cleanup(rendered)` borra los PNG ya escritos y el directorio, y se relanza;
- **siempre**: `document.dispose()` en el `finally`.

Además el llamante vuelve a llamar a `cleanup(pages)` en su propio `finally`. Es idempotente: `cleanup` comprueba `exists()` antes de borrar.

---

4 · Interpretación del contenido

4.1 La cadena completa

`ScanRecord.fromBarcode` es donde convergen las tres piezas:

```
final String raw = payloadForBarcode(barcode);
final SecurityAssessment assessment = ScanSecurityAnalyzer.analyze(raw);
final ParsedContent parsed = ContentParserRegistry.instance.parse(raw);
final String digest = sha256.convert(utf8.encode('$raw|${timestamp.microsecondsSinceEpoch}')).toString().s
```

`payloadForBarcode` resuelve el caso incómodo de un código **binario**: si no hay texto legible pero sí bytes, devuelve `binary-base64:<base64>`. Ese prefijo es el que reconocen después tanto el analizador (`_isKnownStructuredContent`) como el intérprete (rama 1). Es un contrato interno de tres puntos que hay que cambiar a la vez si se toca.

`displayValue` se calcula aparte:

```
final String display = readable.isNotEmpty
   ? readable
   : binaryBytes == null ? raw : 'Datos binarios (${binaryBytes.length} bytes)\n$raw';
```

Es decir, para un código binario el valor mostrado antepone una línea legible al Base64. `rawValue` sigue siendo el Base64 puro, que es lo que se copia y se comparte.

`_decodedBytes` distingue dos tipos concretos con `is`:

```
if (value is DecodedBarcodeBytes) return value.bytes;
if (value is DecodedVisionBarcodeBytes) return value.bytes ?? value.rawBytes;
return null;
```

Está escrito así porque un `switch` sobre esos tipos tenía una rama inalcanzable que el analizador marcaba; el defecto está registrado en `VALIDATION.md`, apartado de correcciones de 1.0.0. `DecodedVisionBarcodeBytes` prioriza `bytes` sobre `rawBytes` porque Apple Vision entrega los dos y el primero ya viene decodificado.

4.2 `_segments`: el analizador de escapes

Es la función que hace posible interpretar un QR de Wi-Fi con puntos y comas o dos puntos dentro del nombre o de la contraseña:

```
for (final int rune in value.runes) {
 final String char = String.fromCharCode(rune);
 if (escaped) { current.write(char); escaped = false; }
 else if (char == r'\') { escaped = true; }
 else if (char == ';') { parts.add(current.toString()); current.clear(); }
 else { current.write(char); }
}
```

Máquina de estados de dos estados sobre puntos de código, no sobre unidades UTF-16: `value.runes` evita partir un carácter fuera del plano básico. Después, cada parte se divide por el **primer** `:`, de modo que un valor que contenga dos puntos sin escapar no rompe la clave.

Hay prueba dedicada con el caso exacto: `WIFI:T:WPA;S:Mi\;Red;P:clave\:segura;H:false;;` debe producir `Red = Mi;Red` y `Contraseña = clave:segura`.

4.3 `_tlv`: EMVCo sin lanzar

```
while (index + 4 <= raw.length) {
 final String id = raw.substring(index, index + 2);
 final int? length = int.tryParse(raw.substring(index + 2, index + 4));
 if (length == null || index + 4 + length > raw.length) break;
 result[id] = raw.substring(index + 4, index + 4 + length);
 index += 4 + length;
```

```
}
```

`break` en vez de excepción: un QR de pago truncado o mal formado devuelve los campos que sí se pudieron leer, en lugar de perderse por completo. El resultado siempre incluye el campo `Carga` con el texto íntegro, para que el usuario pueda verificarlo aunque la interpretación sea parcial.

4.4 Los formatos de pago por posición de línea

Swiss QR Bill y EPC/SEPA se interpretan **por número de línea**:

```
if (lines.length > 3) 'IBAN': lines[3],
if (lines.length > 5) 'Nombre': lines[5],
if (lines.length > 18) 'Monto': lines[18],
```

Es lo que especifican ambos estándares —son formatos posicionales— pero significa que una variante de versión con líneas insertadas produciría campos desplazados sin ningún aviso. Los `if (lines.length > N)` evitan el desbordamiento, no el desalineamiento. R-18.

Los cuatro tipos de pago y el documento AAMVA marcan `sensitive: true`, lo que los deja fuera del historial automático y hace que sus campos aparezcan enmascarados si `hideSensitiveValues` está activo.

4.5 Cómo decide `ScanRecordCard` qué enmascarar

```
final bool conceal = record.isSensitive && widget.settings.value.hideSensitiveValues && !_revealed;
…
final bool sensitiveField = record.parsed.sensitive &&
   <String>{'Contraseña', 'Secreto', 'Dirección', 'IBAN', 'Carga'}.contains(field.key);
final String value = conceal && sensitiveField ? '••••••••' : field.value;
```

El enmascaramiento es **por campo**, con una lista de cinco nombres. Un registro sensible muestra el resto de sus campos en claro: en un QR de Wi-Fi el nombre de la red se ve y la contraseña no. El botón del ojo alterna `_revealed`, que es estado local del widget y **no se persiste**.

Consecuencia que conviene saber: un campo nuevo con datos sensibles y otro nombre —por ejemplo `Clave` o `Semilla`— **no se enmascararía**. La lista es literal. R-19.

---

5 · Cifrado y persistencia

5.1 El sobre

```
return jsonEncode(<String, Object?>{
 'version': currentVersion,
 'algorithm': currentAlgorithm,
 'keyId': selectedKeyId,
 'createdAt': DateTime.now().toUtc().toIso8601String(),
 'payload': <String, String>{
   'nonce': base64Encode(box.nonce),
   'cipherText': base64Encode(box.cipherText),
   'mac': base64Encode(box.mac.bytes),
 },
});
```

Los cuatro campos de cabecera son **metadatos en claro** y están ahí a propósito: sin ellos, un descifrador futuro no podría decidir con qué llave ni con qué algoritmo intentar. `createdAt` no se usa en ninguna decisión del código —**comprobado**: no aparece leído en ningún sitio— y existe como información forense.

El sobre heredado es simplemente el objeto `payload` suelto. La detección es la ausencia de `version`, no una versión `1` explícita, porque el formato inicial no escribía ningún número.

5.2 La doble barrera de la llave ausente

```
final SecretKey? key = await _keyProvider.read(keyId);
if (key == null) {
 _missingKeyIds.add(keyId);
 throw StateError('encryption_key_missing:$keyId');
}
_missingKeyIds.remove(keyId);
```

y, en `encryptJson`:

```
if (_missingKeyIds.contains(selectedKeyId)) {
 throw StateError('encryption_key_missing:$selectedKeyId');
}
```

Es la propiedad más importante del cifrador. Sin la segunda comprobación: `decryptJson` fallaría, el repositorio registraría la incidencia, y la siguiente escritura llamaría a `readOrCreate`, que **crearía una llave nueva bajo el mismo identificador**. A partir de ahí los registros antiguos serían irrecuperables incluso si la llave original reapareciera, porque el almacén ya tendría otra.

La cuarta línea —`_missingKeyIds.remove(keyId)` tras un descifrado correcto— es la salida: si la llave vuelve a aparecer (por ejemplo tras una restauración del sistema), el bloqueo se levanta solo.

Hay una prueba que recorre exactamente este camino: `test/core/payload_cipher_test.dart`, «a missing decryption key is never replaced silently».

5.3 Actualización perezosa de sobres

Ambos repositorios hacen lo mismo tras un descifrado correcto:

```
final CipherEnvelopeInfo info = _cipher.inspect(encrypted);
if (info.legacy || info.version < PayloadCipher.currentVersion) {
 await _store.record(snapshot.key).update(_database.database, <String, Object?>{
   'payload': await _cipher.upgradeEnvelope(encrypted),
 });
}
```

`inspect` no descifra, así que la comprobación es barata y ocurre en cada lectura. La reescritura solo cuando hace falta.

Cuidado: la condición **no compara el `keyId`**. `upgradeEnvelope` sí lo hace (`info.keyId == _activeKeyId`), pero como aquí no se llama salvo que la versión sea antigua, un registro cifrado con una llave anterior pero versión actual **no se actualiza al leerlo**. Lo actualiza la rotación completa, no la lectura. Es coherente —rotar es una operación explícita— pero no está documentado.

5.4 Migración del historial desde preferencias

Es el flujo con más precauciones de todo el repositorio, y por buenas razones: convierte un historial en claro guardado en preferencias a la base cifrada, y un fallo a mitad podría perder datos del usuario.

```
final String encryptedBackup = await _cipher.encryptJson(<String, Object?>{
 'schemaVersion': 1,
 'encoding': 'utf8-base64',
 'raw': base64Encode(utf8.encode(stored)),
});
await _preferences.setString(_legacyBackupKey, encryptedBackup);
```

Paso 1: respaldo cifrado del origen exacto, antes de tocar nada. El comentario lo dice: «Preserve the exact legacy source, but never duplicate it in clear text». Se guarda el texto original en Base64 dentro de un sobre cifrado, no los registros reinterpretados: si la reinterpretación fuera la que falla, el respaldo sigue siendo fiel.

```
await _migrateRecordsAtomically(records);
```

Paso 2: inserción con verificación dentro de la transacción.

```
final Set<String> before = await ids();
final Set<String> expected = <String>{...before, ...records.map((r) => r.id)};
if (expected.length > maxItems) throw StateError('migration_limit_exceeded');
// … precifrado fuera de la transacción …
await _database.database.transaction((transaction) async {
 for (…) { await _store.record(…).put(transaction, …); }
 final stored = await _store.find(transaction);
 final actual = stored.map((i) => i.key).toSet();
 if (!actual.containsAll(expected)) throw StateError('migration_verification_failed');
});
```

La verificación ocurre **dentro** de la transacción, así que lanzar la revierte por completo. Y el cifrado se precalcula fuera para no mantener la base bloqueada durante miles de operaciones AES.

```
await _preferences.remove(_legacyStorageKey);
await _preferences.remove(_migrationErrorKey);
await _preferences.setBool(_migrationKey, true);
```

Paso 3: solo entonces se borra el origen.

Y si algo falla:

```
} on Object catch (error) {
 final String code = 'legacy_${error.runtimeType}';
 await _preferences.setString(_migrationErrorKey, code);
 await _recovery.record(entityType: RecoveryEntityType.migration, entityId: 'history_v1_to_v2', code: code
 …
 return HistoryMigrationStatus(completed: false, errorCode: code, backupPresent: backupPresent);
}
```

El origen queda intacto, la conversión **no se marca completada** (así que se reintentará en el siguiente arranque), se crea una incidencia en el Centro de recuperación y Ajustes muestra la opción «Reintentar migración del historial».

En una instalación limpia todo esto se resuelve en dos líneas: no hay `scan_history_v1`, se marca completada y se devuelve.

5.5 Recorte del historial

```
Future<void> _trim() async {
 final int count = await _store.count(_database.database);
 if (count <= maxItems) return;
 final old = await _store.find(_database.database,
   finder: Finder(sortOrders: [SortOrder('scannedAt')], limit: count - maxItems));
 await _database.database.transaction((t) async {
   for (final snapshot in old) await _store.record(snapshot.key).delete(t);
 });
}
```

Ordena **ascendente** por `scannedAt` (`SortOrder('scannedAt')` sin `false`) y borra exactamente el excedente: los más antiguos. El campo en claro `scannedAt` existe precisamente para poder hacer esto sin descifrar 5000 registros.

---

6 · Rotación de llave

`DataMaintenanceService.rotateEncryptionKey` merece leerse entera porque su corrección depende del orden.

```
if (_rotationInProgress) throw StateError('encryption_rotation_in_progress');
_rotationInProgress = true;
final String keyId = 'v3_${DateTime.now().toUtc().millisecondsSinceEpoch}';
```

La bandera es la protección contra reentrada. La interfaz también la tiene (`_rotatingKey` deshabilita la fila), pero la clase no confía en la interfaz.

El identificador incluye la marca de tiempo, así que dos rotaciones producen llaves distintas y ninguna sobrescribe a la anterior en el almacén seguro. *Inferencia*: las llaves antiguas **no se borran** tras una rotación exitosa; quedan en el Keychain o Keystore bajo su propio `ucs_database_key_v3_…`. Es prudente (permite recuperar un registro que se hubiera quedado atrás) pero significa que el almacén crece una entrada por rotación. R-20.

```
for (final item in history) {
 final String? payload = item.value['payload'] as String?;
 if (payload == null) throw StateError('history_payload_missing:${item.key}');
 historyPayloads[item.key] = await _cipher.encryptJson(await _cipher.decryptJson(payload), keyId: keyId);
}
```

Un registro sin `payload` **aborta la rotación completa** en vez de saltárselo. Es la decisión correcta: rotar dejando registros atrás produciría una base con dos llaves activas de facto. Hay prueba dedicada («rotation aborts before changing metadata when a payload is incomplete»).

```
await _database.database.transaction((transaction) async {
 for (…) await _history.record(…).update(transaction, …);
 for (…) await _inventory.record(…).update(transaction, …);
 await _metadata.saveActiveKeyId(keyId, transaction: transaction);
});
_cipher.activateKey(keyId);
```

Los datos y el metadato de llave activa se escriben en **la misma transacción**. Si se separaran, un fallo entre ambas dejaría la base con datos nuevos y llave declarada vieja, o al revés. Y `activateKey` —que solo cambia una variable en memoria— ocurre **después** de que la transacción confirme.

```
} on Object {
 try { await _cipher.forgetKey(keyId); } on Object { }
 rethrow;
} finally {
 _rotationInProgress = false;
}
```

El `catch` limpia la llave huérfana. El `try` anidado y vacío está ahí para que un fallo al borrar **no sustituya al error original de la rotación**, que es el que el usuario necesita ver. El comentario del código lo explica. Hay prueba dedicada («failed rotation removes a newly generated orphan key»).

---

7 · Inventario continuo

7.1 La cola serial

Nueve métodos de `InventoryStore` pasan por `_writes.run`. El patrón es siempre el mismo:

```
Future<void> addScan(ScanRecord record) => _writes.run<void>(() async {
     final InventorySession? session = activeSession;
     if (session == null) return;
     …
```

```
    await _replaceUnlocked(session.copyWith(items: items));
  });
```

`_replaceUnlocked` se llama así porque **asume que ya está dentro de la cola**. Llamarlo desde fuera rompería la garantía. No hay nada en el código que lo impida; el nombre es toda la protección. *Inferencia*: es una convención, no un mecanismo.

La razón de la cola es concreta: el inventario recibe lecturas continuas de la cámara mientras el usuario edita cantidades y notas. `InventorySession` es inmutable, así que dos escrituras concurrentes leerían el mismo estado base y la segunda perdería el cambio de la primera.

`AsyncWriteQueue` tiene una propiedad que merece nombrarse:

```
_tail = _tail.then<void>((_) async {
 try { completer.complete(await operation()); }
 on Object catch (error, stack) { completer.completeError(error, stack); }
});
```

El error se entrega al `Completer` del llamante, pero **la cadena `_tail` siempre resuelve con éxito**. Un fallo no envenena la cola. Hay prueba dedicada.

7.2 El conteo

```
final InventoryItem? existing = items[record.rawValue];
items[record.rawValue] = existing == null
   ? InventoryItem(code: record.rawValue, format: record.format,
       label: record.parsed.summary ?? record.contentType,
       quantity: 1, firstScannedAt: now, lastScannedAt: now)
   : existing.copyWith(quantity: existing.quantity + 1, lastScannedAt: now);
```

La clave del mapa es el **texto bruto del código**. La etiqueta se toma de `parsed.summary` si existe (el dominio de una URL, el nombre de un contacto, el GTIN de un GS1) y si no del tipo de contenido.

`setQuantity(code, 0)` o negativo **elimina el ítem** del mapa. Es lo que hace «Quitar del inventario», y también lo que hace pulsar «restar» sobre un ítem con una unidad.

7.3 Lo que el inventario no filtra

`_onDetect` de `InventoryScreen` llama a `widget.store.addScan(record)` **sin comprobar `record.isSensitive`**. Un QR de Wi-Fi con contraseña, un OTP o un QR de pago escaneado en una sesión de inventario **se persiste** —cifrado, como todo lo demás— con su texto íntegro como clave del ítem.

Es una asimetría real con `_persistAndShow`, que sí filtra. Registrada como R-03. No es una fuga: el dato está cifrado con la misma llave que el historial. Pero contradice la garantía enunciada en `docs/SECURITY.md` punto 5 («No persistir automáticamente OTP ni Wi-Fi con contraseña») y en la política de privacidad.

7.4 La cámara del inventario

`InventoryScreen` **no usa `ScannerEngine`**. Construye su propio `MobileScannerController` y lo maneja directamente. Sus opciones difieren de las de `MobileScannerEngine`:

Opción	`MobileScannerEngine`	`InventoryScreen`
`detectionSpeed`	`noDuplicates`	`noDuplicates`
`formats`	`[]` (todos)	`[]` (todos)
`invertImage`	`true`	`true`

`autoZoom`	`true`	`true`
`torchEnabled`	`_settings.autoTorch`	**no se pasa**
`returnImage`	`false`	**no se pasa**

`autoTorch` no se aplica en el inventario. Es una diferencia de comportamiento observable y no documentada. R-04.

Su marco de lectura es un rectángulo horizontal (`width: maxWidth.clamp(180,260)`, `height: 120`) en lugar del cuadrado del escáner, porque los códigos de producto son lineales y anchos.

Y su ventana antirrebote es de **1200 ms sobre un solo código**, no de 2 s sobre la combinación: en un inventario, escanear dos veces el mismo producto debe poder sumar dos unidades sin esperar dos segundos.

---

8 · Recuperación

8.1 Cuándo se crea una incidencia

Ambos repositorios registran en dos casos: `payload` ausente (`code: 'missing_payload'`) y descifrado fallido (`code: 'decrypt_<Tipo>'`, con la carga cifrada guardada). La migración registra un tercero (`legacy_<Tipo>`).

El `id` de la incidencia es determinista:

```
final String source = '${entityType.name}|$entityId|$code';
final String id = sha256.convert(source.codeUnits).toString().substring(0, 24);
```

Lo cual significa que **la misma incidencia detectada dos veces no se duplica**: la segunda sobrescribe a la primera con estado `unresolved`. Es lo que evita que cada arranque llene la lista con la misma entrada. El efecto lateral es que marcar «revisado» un problema recurrente lo devuelve a la lista en la siguiente lectura.

8.2 `retry`

```
final Map<String, dynamic> clear = await _cipher.decryptJson(encrypted);
final String upgraded = await _cipher.upgradeEnvelope(encrypted);
switch (issue.entityType) {
 case RecoveryEntityType.history:
   final ScanRecord record = ScanRecord.fromJson(clear);
   await _history.record(issue.entityId).put(…, {'id': record.id, 'scannedAt': …, 'payload': upgraded});
```

Descifra **y reconstruye el modelo** antes de reinsertar. Si el JSON descifrado no fuera un `ScanRecord` válido, `fromJson` lanza y el `catch` global devuelve `false`: no se reinserta un registro corrupto.

Nótese que se reinserta con `record.id` recalculado del JSON, no con `issue.entityId`. En un caso normal coinciden.

El `catch` de `retry` es `on Object { return false; }`: cualquier fallo se convierte en un `false` que la interfaz traduce a «No fue posible recuperar el elemento con la llave disponible». Es correcto para el usuario, pero **se pierde la causa**: no se registra en `AppDiagnostics`. R-21.

8.3 `exportBundle`

```
'notice': 'Encrypted payloads remain encrypted. No decryption key is included.',
'issues': issues.map((i) => i.toJson()).toList(growable: false),
```

El paquete contiene los sobres **tal cual**, todavía cifrados. Hay una prueba que lo verifica: cifra un objeto con la palabra `never-export-clear`, genera el paquete y comprueba que ni esa palabra ni la cadena `ucs_database_key`

aparecen.

---

9 · Generador

Flujo simple: nueve tipos de carga y diez formatos visuales, ambos como enumeraciones privadas. `_payload` construye la cadena según el tipo, y `BarcodeWidget` la dibuja.

Dos detalles merecen mención.

El escape del Wi-Fi:

```
static String _escape(String value) => value
   .replaceAll(r'\', r'\\')
   .replaceAll(';', r'\;')
   .replaceAll(':', r'\:')
   .replaceAll(',', r'\,');
```

El orden importa: la barra invertida se escapa **primero**, o se escaparían las barras que la propia función acaba de introducir. Es la operación inversa de `ContentInterpreter._segments`, así que un QR generado por esta pantalla se reinterpreta correctamente al escanearlo.

La normalización de URL:

```
static String _normalizeUrl(String value) =>
   RegExp(r'^https?://', caseSensitive: false).hasMatch(value.trim())
       ? value.trim() : 'https://${value.trim()}';
```

Antepone `https://`, nunca `http://`.

El PNG se produce capturando el `RepaintBoundary` a `pixelRatio: 3`; el SVG lo genera `barcode_widget` a resoluciones fijas (900×900 para 2D, 1200×420 para lineales). Ambos se comparten con `share_plus` sin escribir a disco: van como `XFile.fromData`.

`errorBuilder` de `BarcodeWidget` cubre el caso de un contenido inválido para el formato elegido —por ejemplo, letras en un EAN-13— con un mensaje explícito.

---

10 · Exportación

```
static Future<void> _shareCsv(List<List<String>> rows, String name, String title) async {
 final Uint8List bytes = Uint8List.fromList(await compute<…>(_encodeCsv, rows));
 await SharePlus.instance.share(ShareParams(title: title, files: [XFile.fromData(bytes, mimeType: 'text/cs
}
```

`compute` mueve la codificación a otro isolate. Con 5000 registros, generar el CSV en el isolate principal produciría un salto visible en la interfaz.

`_encodeCsv` y `_encodeXlsx` son **funciones de nivel superior**, no métodos estáticos de la clase: `compute` exige un punto de entrada de nivel superior o estático, y en Dart 3 una función de nivel superior es la forma que siempre funciona.

```
List<int> _encodeCsv(List<List<String>> rows) {
 String csvCell(String value) => '"${value.replaceAll('"', '""')}"';
 final String csv = rows.map((row) => row.map(csvCell).join(',')).join('\r\n');
 return <int>[0xEF, 0xBB, 0xBF, ...utf8.encode(csv)];
}
```

Tres decisiones que solo tienen sentido si el destino es Excel en Windows: **todas** las celdas entrecomilladas (no solo las que lo necesitan), `\r\n` como separador de línea, y el BOM UTF-8 al principio. Sin el BOM, Excel abre el archivo en la página de códigos del sistema y los acentos salen rotos.

`_encodeXlsx` crea la hoja con el nombre pedido y **borra la hoja por defecto** si el nombre difiere:

```
final String firstSheet = workbook.getDefaultSheet() ?? 'Sheet1';
final Sheet sheet = workbook[sheetName];
…
if (firstSheet != sheetName) workbook.delete(firstSheet);
```

Sin esa última línea, cada archivo llevaría una hoja «Sheet1» vacía junto a la buena.

---

11 · Pantallas resumidas

`HistoryScreen` (284 líneas). Filtra en memoria sobre `store.history` por texto —que busca en `rawValue`, `contentType`, `format`, etiquetas y notas—, por nivel de riesgo y por favoritos. Se suscribe al store con `addListener` y llama a `setState` vacío. Exporta e importa desde un menú. El diálogo de importación muestra la vista previa (válidos, duplicados, rechazados) **antes** de ofrecer las tres estrategias. Nada se escribe hasta elegir una.

`GeneratorScreen` (310 líneas). Ver el apartado 9.

`FormatsScreen` (170 líneas). Puramente informativa: tres grupos con 20 nombres de formato escritos a mano y dos tarjetas de texto. No consulta al motor ni a `BarcodeLabels`.

`SettingsScreen` (378 líneas). Cinco secciones de `SwitchListTile`, `DropdownButton` y `ListTile`. Las tres operaciones con consecuencias —rotar llave, borrar historial, reintentar migración— tienen diálogo de confirmación. `_rotateEncryptionKey` usa el mismo patrón de diálogo bloqueante que `_runBatchDialog`, con `dialogOpen` y `navigator` capturado por adelantado.

`RecoveryScreen` (139 líneas). `FutureBuilder` sobre `service.load(unresolvedOnly: false)` —muestra también las resueltas— con un menú de cuatro acciones por incidencia. Solo ofrece «Reintentar recuperación» si `issue.encryptedPayload != null`, y «Reintentar migración» solo para incidencias de tipo `migration`.

---

12 · Lo que un lector nuevo descubriría a golpes

Recopilado de todo lo anterior, para no tener que releerlo:

1. **Un clon no compila.** Falta `python tool/bootstrap.py`.
2. **Leer el historial escribe en la base**, si algún sobre está desactualizado.
3. **El momento de cada lectura es visible sin la llave**: `scannedAt` va en claro para poder ordenar y podar.
4. **Un registro que no descifra no rompe la carga**; se omite y se registra.
5. **El inventario sí guarda contenido sensible**; el historial no.
6. **`autoTorch` no se aplica en el inventario.**
7. **El temporizador del portapapeles sobrevive al widget** que lo creó, y comprueba que el contenido no haya cambiado antes de borrar.
8. **Los diagnósticos se pierden al cerrar**: viven solo en memoria.
9. **Las llaves antiguas no se borran** tras una rotación exitosa.
10. **`restablecer preferencias visuales` no toca** `save_history`, `private_mode` ni `biometric_lock`.

11. **Desinstalar borra el historial para siempre**: la llave se va con la aplicación y no hay copia remota porque no hay servidor.
12. **Lo exportado va en claro.** Un CSV del historial es un archivo normal.
13. **Un EAN-13 que empiece por 978 o 979 se clasifica como ISBN**, no como producto.
14. **La lista de campos que se enmascaran es literal**: `Contraseña`, `Secreto`, `Dirección`, `IBAN`, `Carga`.

Continúa en 07 · Base de datos.